Taken together, partials and layouts provide an effective way to make sure that the user interface portion of your application is maintainable. But being maintainable is only part of the story; doing so in a way that also performs well is also crucial.

What We Just Did

Views are the public face of Rails applications, and we've seen that Rails delivers extensive support for what you need to build robust and maintainable user and application programming interfaces.

We started with templates, of which Rails provides built-in support for three types: ERB, Builder, and SCSS. Templates make it easy for us to provide HTML, JSON, XML, CSS, and JavaScript responses to any request. We'll discuss adding another option in *Creating HTML Templates with Slim*.

We dove into forms, which are the primary means by which users will interact with your application. Along the way, we covered uploading files.

We continued with helpers, which enable us to factor out complex application logic to allow our views to focus on presentation aspects. We explored a number of helpers that Rails provides, ranging from basic formatting to hypertext links, which are the final way in which users interact with HTML pages.

We completed our tour of Action View by covering two related ways of factoring out large chunks of content for reuse. We use layouts to factor out the outermost layers of a view and provide a common look and feel. We use partials to factor out common inner components, such as a single form or table.

That covers how a user with a browser will access our Rails application. Next up: covering how we define and maintain the schema of the database our application will use to store data.